UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

Old World

Autor: Bodea George

Facultatea de Automatica
si Calculatoare

An universar: 2025-2026

Cuprins

1 Prezentarea temei **2**

2 Scenariul **2**
2.1 Descrierea scenei și a obiectelor 2
2.2 Funcționalități 3

3 Detalii de implementare **3**
3.1 Funcții și algoritmi 3
3.1.1 Soluții posibile 5
3.1.2 Motivarea abordării alese 5
3.2 Modelul grafic 5
3.3 Structuri de date 6
3.4 Ierarhia de clase 7

4 Manual de utilizare **7**

5 Concluzii și dezvoltări ulterioare **8**

6 Referințe **8**

1 Prezentarea temei

Tema proiectului a constat în crearea și implementarea unei scene 3D folosind OpenGL. Scena trebuia să includă elemente de bază precum texturi, lumini și umbre. În rest, putea conține orice alte elemente considerate adecvate pentru scena modelată, atâta timp cât acestea contribuiau la îmbunătățirea fotorealismului scenei și a obiectelor din aceasta.

2 Scenariul

2.1 Descrierea scenei și a obiectelor

Scena prezintă o parte din insula *The Wandering Isle*, care este prezentă în jocul *World of Warcraft*. Astfel, scena încearcă să simuleze din punct de vedere vizual această insulă din joc. Porțiunea de insulă modelată face parte dintr-un sat locuit de urși panda.

Printre obiectele prezente în scenă se numără casele specifice ursilor panda, copacii, lămpile și alte obiecte caracteristice unui sat. Piesa de rezistență a scenei este cel mai mare templu de pe insulă, numit *Temple of Five Dawns*, către care converg toate drumurile din zonă.

În această scenă se poate controla unul dintre ursii panda, care se poate deplasa liber și poate explora insula într-un mod liniștit și relaxant.

Mai jos se poate observa o comparație între scena de obiecte din jocul *World of Warcraft* și propria scenă de obiecte:

Figura 1: Scena de obiecte din jocul *World of Warcraft*

Figura 2: Propria scenă de obiecte

2.2 Funcționalități

Proiectul include o animație de vizualizare prin care este prezentat obiectivul principal al insulei, *Temple of Five Dawns*. Această animație permite observarea templului dintr-o perspectivă diferită față de cea disponibilă în timpul explorării satului cu ursul panda.

Utilizatorul poate explora insula într-un mod similar unui joc, admirând toate zonele și detaliile acesteia. De asemenea, în cazul în care ursul panda obosește după o perioadă îndelungată de deplasare, acesta se poate opri și poate executa anumite acțiuni.

Aceste acțiuni sunt declanșate prin introducerea unor cuvinte-cheie de la tastatură, precum *EmoteCheer*, iar ursul panda va executa comanda primită prin intermediul unei animații corespunzătoare.

Funcționalitățile principale ale proiectului sunt:

- Animație cinematică de prezentare a templului *Temple of Five Dawns*;
- Controlul personajului principal (ursul panda) pentru deplasare liberă în scenă;
- Explorarea mediului 3D într-un mod interactiv;
- Sistem de comenzi introduse de la tastatură;
- Execuția de animații specifice în funcție de comenzile primite (ex. *EmoteCheer*);
- Oprirea personajului și interacțiunea cu mediul înconjurător.

3 Detalii de implementare

3.1 Funcții și algoritmi

O funcție esențială în cadrul proiectului este cea responsabilă de includerea obiectelor în scenă. Aceasta se bazează pe o abordare recursivă, pornind de la teren și continuând până la cele mai mici obiecte, într-o manieră ierarhică. Această abordare a fost necesară deoarece fișierele asociate obiectelor conțin un fișier de tip *CSV* în care sunt definiți toți copiii unui obiect.

De exemplu, dacă într-o clădire există un scaun, acesta are poziția, rotația și scalarea definite relativ față de obiectul părinte, în acest caz clădirea. Astfel, transformările sunt propagate corect în cadrul ierarhiei scenei.

Un algoritm important utilizat în proiect este cel pentru detectarea coliziunilor dintre ursul panda și celelalte obiecte din scenă. În primă fază, este utilizată o structură ierarhică de tip *Bounding Volume Hierarchy* (BVH), în care sunt adăugate toate obiectele din scenă, inclusiv personajul principal, pe baza volumelor de încadrare de tip *Axis-Aligned Bounding Box* (AABB) [**bvh**].

Prin utilizarea acestei structuri, nu mai este necesară verificarea intersecției dintre toate obiectele din scenă. Dacă ursul panda nu se intersectează cu un anumit volum AABB superior, atunci este garantat că nu se va intersecta nici cu volumele copil ale acestuia. După identificarea volumelor AABB cu care ursul panda se intersectează, se aplică un algoritm mai precis, numit *Separating Axis Theorem* (SAT) [**sat**]. Acesta verifică intersecția dintre primitivele geometrice ale obiectului și AABB-ul ursului panda, iar în cazul în care se detectează o intersecție, cele două obiecte sunt considerate în coliziune.

Un alt algoritm utilizat este cel pentru realizarea animațiilor personajului principal, bazat pe metoda *Skeletal Animation* [**skeletal**]. Această tehnică permite manipularea individuală a fiecărui os din structura scheletală a modelului ursului panda, oferind impresia unei animații fluide și realiste.

Un avantaj major al animației scheletale este relația ierarhică dintre oasele părinte și copil. De exemplu, la rotirea umărului drept, se vor deplasa în mod corespunzător bicepsul, antebrațul, mâna și degetele asociate acestuia.

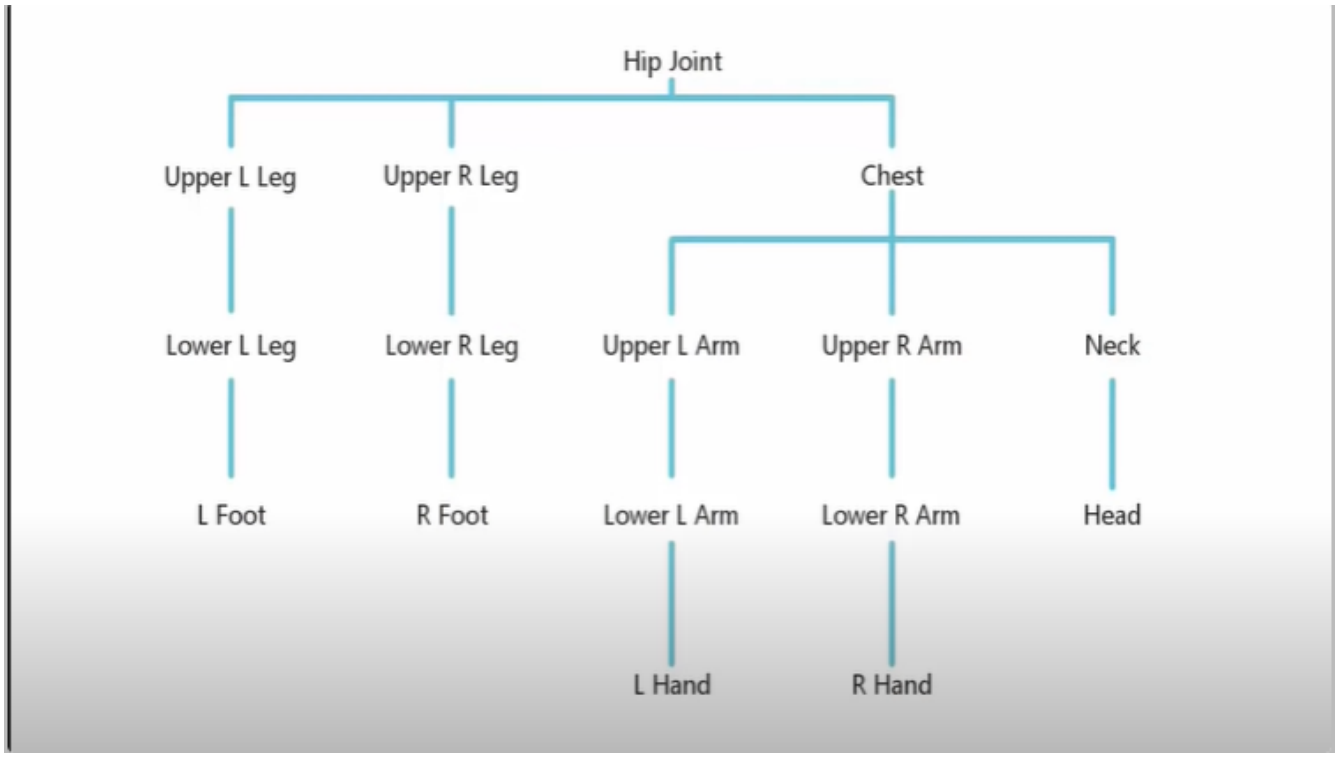

Figura 3: Un exemplu de ierarhie de oase care ar putea modela un obiect

După definirea acestei ierarhii, animațiile sunt realizate prin utilizarea unor *keyframe-uri*, care reprezintă poziții discrete ale scheletului pe parcursul animației. Prin interpolarea dintre aceste cadre-cheie se obține animația finală.

3.1.1 Soluții posibile

O posibilă alternativă pentru construirea scenei ar fi fost crearea unui singur obiect de dimensiuni mari, care să conțină întreaga scenă (cu excepția personajului panda), folosind o aplicație de modelare 3D precum *Blender*. Această abordare ar fi fost mai simplu de implementat din punct de vedere al încărcării obiectelor și al gestionării ierarhiei scenei, însă ar fi redus flexibilitatea în ceea ce privește modificarea sau extinderea ulterioară a scenei.

În ceea ce privește detectarea coliziunilor, o altă soluție posibilă ar fi fost verificarea intersecției dintre volume de tip *Oriented Bounding Box* (OBB). Această metodă ar fi redus numărul de rezultate fals pozitive, deoarece volumele OBB nu sunt aliniate cu axele sistemului de coordonate și nu trebuie să se extindă excesiv atunci când un obiect este rotit pentru a-l încadra complet.

Cu toate acestea, această abordare ar fi adus un grad suplimentar de complexitate, întrucât ar fi necesitat construirea și gestionarea unei ierarhii de tip *Bounding Volume Hierarchy* bazată pe OBB-uri. Implementarea unei astfel de structuri ar fi fost mai dificilă și mai costisitoare din punct de vedere computațional comparativ cu utilizarea AABB-urilor.

Pentru realizarea animațiilor, o altă abordare ar fi constat în utilizarea transformărilor de bază — rotație, translație și scalare — aplicate direct asupra diferitelor părți ale corpului ursului panda (torso, umeri, cap etc.). Această metodă ar fi fost mai puțin flexibilă, deoarece s-ar fi pierdut relația ierarhică părinte–copil dintre componentele corpului. În plus, animațiile ar fi fost mai greu de realizat și mai puțin fluide, fiind necesară definirea manuală a transformărilor pentru fiecare componentă în parte.

3.1.2 Motivarea abordării alese

Abordarea ierarhică pentru construirea scenei a fost aleasă din mai multe motive. În primul rând, aceasta a simplificat considerabil implementarea sistemului de coliziuni, permițând verificarea eficientă a intersecțiilor dintre volumele de tip AABB prin intermediul unei structuri ierarhice, așa cum a fost descris anterior. În al doilea rând, această organizare a facilitat identificarea și gestionarea surselor de lumină din scenă, acestea putând fi extrase și salvate separat chiar în momentul încărcării modelelor, pe baza denumirilor obiectelor.

Pentru detectarea coliziunilor a fost aleasă o combinație între verificarea la nivel de volum de încadrare (AABB) și verificarea precisă la nivel de primitive geometrice. Coliziunile bazate exclusiv pe AABB nu ar fi oferit o precizie suficientă, acestea fiind utilizate doar ca o etapă de filtrare (*broad-phase*). Astfel, verificarea suplimentară la nivel de triunghi, utilizând un algoritm mai costisitor din punct de vedere computațional, precum *Separating Axis Theorem* (SAT), a fost necesară pentru a obține rezultate corecte și realiste.

În ceea ce privește animațiile, a fost aleasă metoda *Skeletal Animation* datorită necesității obținerii unor mișcări fluide și realiste, similare celor întâlnite în jocurile comerciale. Având la dispoziție modele care conțineau deja structura scheletală și animațiile predefinite, această abordare a oferit un echilibru optim între timpul de implementare și calitatea finală a animațiilor obținute.

3.2 Modelul grafic

Pentru realizarea modelului grafic au fost utilizate cele două tipuri fundamentale de shadere: *vertex shader* și *fragment shader*. Proiectul a necesitat însă mai multe programe de tip shader, fiecare fiind adaptat unor situații specifice din cadrul scenei (randarea obiectelor statice, animația personajului, afișarea cerului etc.).

Shaderul *worldShader.frag* conține codul aferent fragment shaderului și este responsabil de calculul iluminării scenei. Acesta implementează un model de iluminare de tip Phong, luând în considerare atât lumini direcționale, cât și lumini punctiforme. În scenă sunt utilizate un număr de 54 de surse de lumină, care contribuie la componentele de iluminare ambientală și difuză. De asemenea, shaderul realizează generarea umbrelor folosind o textură de tip *shadow map*, obținută prin randarea scenei din perspectiva sursei de lumină.

Shaderul *shaderStart.vert* conține codul necesar pentru vertex shaderul personajului principal (ursul panda). În cadrul acestuia se realizează calculul poziției fiecărui vârf în funcție de matricele asociate oaselor din structura scheletală. Transformările sunt ponderate folosind coeficienții de influență (*weights*), permițând deformarea corectă a modelului în timpul animațiilor.

Un alt shader important este *SkyBoxShader*, utilizat pentru randarea cerului. Acesta mapează o textură de tip *samplerCube* pe un cub care înconjoară întreaga scenă, creând impresia apartenenței obiectelor la o lume vastă și coerentă.

Transformările geometrice sunt realizate prin utilizarea matricelor de model, vizualizare și proiecție, iar testul de adâncime este activat pentru asigurarea unei randări corecte a obiectelor în spațiul tridimensional.

3.3 Structuri de date

Proiectul utilizează o varietate de structuri de date, începând de la vectori pentru stocarea vârfurilor, normalelor și coordonatelor de textură ale fiecărui obiect din scenă, până la o structură ierarhică care permite salvarea eficientă a relațiilor părinte–copil dintre obiectele din scenă. Această ierarhie este implementată printr-un vector de obiecte, fiecare obiect conținând referințe către copii și către obiectul părinte, facilitând astfel operații recursive precum randarea sau calculul transformărilor.

De asemenea, pentru gestionarea animațiilor ursului panda, este utilizat un *hash map* (dicționar) care stochează toate animațiile încărcate la inițializarea scenei. Accesul la o animație se face rapid, pe baza numelui acesteia, cu o complexitate medie de $O(1)$, ceea ce optimizează considerabil timpul de rulare al aplicației în timpul redării animațiilor.

Una dintre structurile esențiale pentru ierarhia de obiecte din scenă este `bvhNode`. Aceasta poate fi descrisă prin următoarele câmpuri principale:

- **AABB** – volumul de încadrare de tip Axis-Aligned Bounding Box pentru nodul respectiv;
- **primitiveCount** – numărul de primitive (triunghiuri) din care nodul este compus;
- **leftFirst** – indexul primului copil al nodului, restul copiilor fiind consecutivi în structură;

Datorită acestei organizări, traversarea ierarhiei pentru randare sau verificarea coliziunilor devine eficientă, reducând semnificativ numărul de verificări necesare la intersecțiile dintre obiecte.

O altă structură importantă este `Joint`, folosită în cadrul animațiilor scheletale. Aceasta stochează:

- matricea de translație, scalare și rotație implicită (valoarea default, atunci când nu este aplicată nicio animație),
- inversa matricei de *bind*, necesară pentru calcularea transformărilor finale ale articulației,
- `parentIndex`, care indică poziția părinteleui în ierarhia scheletului, permițând construirea unui arbore de articulații pentru implementarea corectă a *Skeletal Animation*.

Prin utilizarea acestor structuri, atât ierarhia obiectelor din scenă, cât și cea a scheletului personajului sunt reprezentate eficient, facilitând calculul transformărilor și al coliziunilor, precum și redarea animațiilor într-un mod fluid și realist.

În concluzie, utilizarea acestor structuri de date permite o organizare eficientă a informațiilor în scenă, optimizarea performanței pentru randare și coliziuni, precum și o gestionare facilă a animațiilor și a interacțiunii utilizatorului cu lumea virtuală.

3.4 Ierarhia de clase

Ierarhia de clase a proiectului este complexă, fiind alcătuită din numeroase clase. Mai jos sunt prezentate cele mai importante clase pentru implementarea algoritmilor descriși anterior.

WorldLoader – Această clasă se ocupă cu parsarea fișierelor `CSV` și `JSON` și cu construirea structurii ierarhice a scenei. WorldLoader creează vectori pentru toate obiectele de tip `StaticEntity` și pentru pozițiile luminilor punctiforme din scenă.

StaticEntity – Clasa stochează datele fiecărui obiect fizic din scenă, incluzând modelul 3D, volumul de încadrare AABB, ierarhia BVH și atributele necesare pentru menținerea relației părinte–copil între obiecte. Aceasta este esențială pentru randare și pentru detectarea coliziunilor.

Collision – Această clasă are o referință către `WorldLoader`, pentru a avea acces la obiectele statice din scenă și pentru a verifica coliziunile cu AABB-ul playerului. Printre metodele sale importante se numără:

- `getGroundHeight` – determină, pe baza unui *heightMap*, dacă playerul se află deasupra sau nu a suprafeței și returnează înălțimea terenului la poziția sa;
- `findCollidingAABBs` – parcurge recursiv ierarhia de obiecte și returnează toate AABB-urile care intersecționează poziția playerului.

Clase pentru animații – Pentru implementarea animațiilor scheletale sunt utilizate următoarele clase:

- `Animator` – conține *hashmap-ul* cu toate animațiile curente ale ursului panda, stocate sub formă de `AnimationClip`.
- `AnimationClip` – stochează numele animației, durata acesteia și o listă de `AnimationChannel`.
- `AnimationChannel` – conține vectorii pentru *keyframes* pentru toate tipurile de transformări (translație, rotație, scalare). Acestea sunt folosite pentru a calcula interpolarea liniară și obținerea animației complete.

Această ierarhie de clase permite gestionarea eficientă a scenei, calculul coliziunilor și redarea animațiilor într-un mod organizat și performant, facilitând extensibilitatea proiectului și adăugarea de noi obiecte sau animații în viitor.

4 Manual de utilizare

Comenzile sunt intuitive, similare celor utilizate în majoritatea jocurilor. Astfel, cu ajutorul tastelor **W**, **A**, **S**, **D** utilizatorul poate deplasa ursul panda prin scenă, explorând mediul înconjurător. Tasta **Space** permite efectuarea unui salt, iar mouse-ul este utilizat pentru controlul camerei.

Camera poate fi poziționată în jurul personajului prin menținerea apăsată a butonului stâng al mouse-ului, iar distanța dintre cameră și ursul panda poate fi ajustată cu ajutorul rotiței mouse-ului.

Pentru inițializarea chatului de comenzi se apasă tasta **Enter**, după care se introduce comanda dorită de la tastatură. Comenzile disponibile sunt prezentate mai jos sub formă de listă:

- *Death*;
- *EmoteApplaud*;
- *EmoteCheer*;
- *EmoteChicken*;
- *EmoteDance*;
- *EmoteLaugh*;
- *SpinningKick*;
- *WALean01*.

Scena poate fi vizualizată în trei moduri diferite: solid, *wireframe* sau punctiform, folosind tastele **T**, **Y** și **U**.

5 Concluzii și dezvoltări ulterioare

În concluzie, acest proiect reprezintă un punct de plecare, existând numeroase posibilități de extindere și îmbunătățire. Aplicației i se pot adăuga atât funcționalități vizuale noi, cât și elemente suplimentare de interacțiune între utilizator, ursul panda și lumea virtuală din care acesta face parte.

Din punct de vedere vizual, scena ar putea fi îmbunătățită prin:

- modificarea algoritmului de calcul al umbrelor astfel încât să țină cont de domeniul vizual al camerei;
- implementarea unui sistem de particule pentru simularea focului, vântului și a cascadelor;
- adăugarea unor animații suplimentare, precum *jump* sau *roll*;
- introducerea unor personaje non-jucabile (NPC-uri), pentru a crește realismul și gradul de imersiune al scenei;
- schimbarea poziției soarelui în funcție de perioada zilei, prin implementarea unui ciclu zi/noapte;
- adăugarea efectelor de ploaie.

Din punct de vedere al interacțiunii cu utilizatorul, aplicația ar putea fi extinsă prin:

- implementarea unui *chat box* care să permită vizualizarea în timp real a comenzilor introduse;
- îmbunătățirea sistemului de deplasare și a mecanismelor de coliziune;
- implementarea unui sistem de viață (*health system*) și a unor adversari;
- adăugarea unor abilități de luptă, corelate cu animațiile noi introduse;
- integrarea unor efecte audio pentru crearea unei atmosfere dinamice, diferite în funcție de zona insulei explorate.

Chiar și cu aceste îmbunătățiri care ar putea fi aduse, proiectul poate fi extins și dezvoltat în mod continuu, fără un punct clar de oprire, singurele limite fiind imaginația și timpul disponibil pentru a lucra la această lume virtuală.

6 Referințe

[1] dyn4j, *Separating Axis Theorem*,`https://dyn4j.org/2010/01/sat/`

[2] Jbikker, *How to build a BVH*, https://jacco.ompf2.com/2022/04/13/how-to-build-a-bvh-part-1-basics/

[3] Ankit Singh Kushwah, Skeletal Animation,https://learnopengl.com/Guest-Articles/2020/Skeletal-Animation